

TESTs - Lesson 03 - Syntax and structure

Why is it necessary to call a function (e.g., `setText`) to change a state variable in a Solidity smart contract, rather than directly modifying the variable's value on the blockchain?

- A Direct modification is faster but less secure.
- B Functions govern and manage the data changes, ensuring safety and security.**
- C Direct modification is only allowed for 'private' variables.
- D State variables can only be updated during contract deployment.

In Solidity, how is the value of a state variable within a smart contract typically changed?

- A By directly modifying its storage address on the blockchain.
- B By sending a special Bitcoin transaction.
- C By calling a function specifically designed to update that variable.**
- D Through an external database connection.

When passing strings to Solidity functions, what's the gas benefit of `calldata` vs `memory`, and when should you use each?

- A `calldata` costs more but allows string changes; `memory` is cheaper for read-only strings.
- B `calldata` avoids copying data to memory (cheaper), but strings are read-only. Use `memory` when you need to modify them.**
- C `memory` is for state variables; `calldata` is for function parameters.
- D There is no gas difference; they are interchangeable for strings.

If a state variable in a Solidity contract is declared as `private`, which of the following is true regarding its accessibility in an inheriting contract?

- ☒ A It can be directly accessed and modified by the inheriting contract.
- ☐ B It is not visible or accessible in the inheriting contract.
- ☐ C It can only be accessed through public getter functions in the inheriting contract.
- ☐ D It becomes 'internal' by default in the inheriting contract.

To enable a child contract (`InheritedHelloWorld`) to override a function (`initialText()`) from its parent contract (`HelloWorld`), what two keywords must be used in the respective function definitions, and why?

- A** The parent function must be `private` and the child function `public`.
- B** The parent function must be `external` and the child function `internal`.
- C** The parent function must be `virtual` and the child function `override`.
- D** Neither keyword is necessary; functions are automatically overridden in Solidity.

How would you design a smart contract to minimize gas costs for frequent profile updates (username, bio)?

- A Store profile data in ``storage`` and use ``public`` functions for direct client modifications.
- B Keep profile data in ``storage`` as state variables, but use ``calldata`` for update function parameters to avoid data copying.
- C Use ``assert()`` to validate incoming data since it's the cheapest error handling method.
- D Store all data off-chain with only hashes on-chain to minimize costs.

If a balance update function has excessive gas costs due to wrong data location, which location did you likely use?



- A You likely used `'calldata'`; a more efficient location for temporary calculations is `'stack'`.
- B You likely used `'storage'`; a more efficient location for temporary calculations within a function is `'memory'`.**
- C You likely used `'memory'`; a more efficient location is `'storage'`.
- D You likely used `'stack'`; a more efficient location is `'calldata'`.

How do `'require()'`, `'assert()'`, and `'revert()'` differ in use and gas impact?



- A `'require()'` handles expected conditions and refunds gas; `'assert()'` handles unexpected conditions/bugs and consumes all gas; `'revert()'` is similar to `'require()'` but allows custom error messages.**
- B `'require()'` is for critical errors and halts the entire blockchain; `'assert()'` is for warnings; `'revert()'` is for standard function returns.
- C `'assert()'` saves gas by reverting early; `'require()'` only works for external calls; `'revert()'` is deprecated.
- D All three functions are interchangeable and have the same gas cost implications, only differing in syntax.

What is the primary purpose of using the `'virtual'` and `'override'` keywords in Solidity for functions within an inheritance hierarchy?



- A To change the function's gas cost.
- B To restrict access to the function from external calls.
- C To allow a derived contract to modify the behavior of an inherited function.**
- D To declare a function as immutable.

In Solidity error handling, what is the main distinction between `'require()'` and `'assert()'`?



- A `require()` is used for internal errors, while `assert()` is for external conditions.
- B `require()` refunds unused gas on failure, whereas `assert()` consumes all gas on failure and indicates a serious internal error.**
- C `assert()` is used to prevent reentrancy attacks, while `require()` is for general validation.
- D `require()` can only be used in payable functions, but `assert()` can be used anywhere.